

Trees and model averaging

Audrey Poterie

Office B137 building Yves Coppens

audrey.poterie@univ-ubs.fr

2026-2027 - M2 DSMS, M2 SDD

Chap. 3 : Bagging and Random forests

3.1 Introduction: model averaging

3.1 Introduction: model averaging

Motivation: Decision trees can be simple, but often produce unstable models with weak performance compared to other supervised methods.

→ Use decision trees as building blocks to construct more powerful prediction models → Model averaging.

Model averaging

- **Bagging** [Breiman, 1996]: fit many large trees to bootstrap-resampled versions of the train set, and average the predictions.
- **Random forests** [Breiman, 2001]: a fancier version of bagging that uses features sampling.
- **Boosting** [Freund et al., 1996]: fit many large or small trees to reweighted versions of the train set (no bootstrap resampling) and use a weighted average of the predictions.

3.1 Introduction: model averaging

Motivation: Decision trees can be simple, but often produce unstable models with weak performance compared to other supervised methods.

→ Use decision trees as building blocks to construct more powerful prediction models → Model averaging.

Model averaging

- **Bagging** [Breiman, 1996]: fit many large trees to bootstrap-resampled versions of the train set, and average the predictions.
- **Random forests** [Breiman, 2001]: a fancier version of bagging that uses features sampling.
- **Boosting** [Freund et al., 1996]: fit many large or small trees to reweighted versions of the train set (no bootstrap resampling) and use a weighted average of the predictions.

Some remarks:

- In general (in terms of performance), **Boosting** $\succ$ **Random forests** $\succ$ **Bagging** $\succ$ **single decision tree**.
- Synonym of model averaging: ensemble learning.

3.2 Bagging

3.2 Bagging

What is bagging ?

Principle: Bagging or **Bootstrap Aggregating** averages a given procedure over many samples.

→ Here, we only focus on bagging of CART trees: build lots of large and unpruned CART trees to bootstrap resampled versions of a data set.

→ But note that bagging can be used with any learning procedure (linear regression, neural network, k-NN, etc.)

Two steps: (1) bootstrapping, (2) aggregation.

Motivations:

- To improve model stability and so model accuracy.
- While avoiding overfitting.
 - **⚠** Important since we use large and unpruned trees.
 - *Reminder: very large trees = trees that fit almost perfectly the data, i.e. trees with almost no bias but large variance.*

What is bootstrapping ?

Data available: only one data set $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots (\mathbf{X}_n, Y_n)\}$ to train the model.

→ If we build B CART trees (with $B > 1$) on a same dataset $\mathcal{D}_n$, we get B **identical** trees..

 We need B samples and we cannot create and/or get additional data.

Solution: use bootstrapping = generate "new" data sets by randomly drawing observations (with or without replacement) into the original data set $\mathcal{D}_n$.

→ We say that we generate bootstrap samples.

What is bootstrapping ?

Data available: only one data set $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots (\mathbf{X}_n, Y_n)\}$ to train the model.

→ If we build B CART trees (with $B > 1$) on a same dataset $\mathcal{D}_n$, we get B **identical** trees..

 We need B samples and we cannot create and/or get additional data.

Solution: use **bootstrapping** = generate "new" data sets by randomly drawing observations (with or without replacement) into the original data set $\mathcal{D}_n$.

→ We say that we generate bootstrap samples.

Bootstrap algorithm

- ▷ **Input:** one data set $\mathcal{D}_n$ with 8 observations.

$\mathcal{D}_n$	obs 1	obs 2	obs 3	obs 4	obs 5	obs 6	obs 7	obs 8
-----------------	-------	-------	-------	-------	-------	-------	-------	-------

- ▷ **Do bootstrapping:** repeat independently the following procedure B times: draw randomly and with replacement observations into $\mathcal{D}_n$.

1	obs 2	obs 6	obs 5	obs 5	obs 2	obs 4	obs 8	obs 7
2	obs 6	obs 8	obs 2	obs 4	obs 2	obs 7	obs 8	obs 1
				$\vdots$				
B	obs 7	obs 8	obs 3	obs 7	obs 2	obs 3	obs 3	obs 1

- ▷ **Output:** B bootstrap-resampled versions of the original data set $\mathcal{D}_n \Leftrightarrow B$ "new" data sets made up of observations from $\mathcal{D}_n$.

$$\mathcal{D}_n^1, \dots, \mathcal{D}_n^B.$$

Notation: let $\theta_1, \dots, \theta_b, \dots, \theta_B$ be B independent random variables distributed as θ and independent of the dataset $\mathcal{D}_n$.

→ θ_b represents the generation of the b -th bootstrap sample $\mathcal{D}_n^b$, ($b = 1, \dots, B$).

Important remarks:

- $\theta_1, \dots, \theta_b, \dots, \theta_B$ are identically distributed as θ that means that we use the same resampling strategy to draw all bootstrap sample.
- $\theta_1, \dots, \theta_b, \dots, \theta_B$ are independent that means that the B bootstrap samples are drawn independently.
- $\theta_1, \dots, \theta_b, \dots, \theta_B$ are independent of $\mathcal{D}_n$ that means that the resampling strategy is independent of the data.

Two sampling strategies used for bagging:

Strategy 1) Draw with replacement $a_n = n$ observations into $\mathcal{D}_n$, ($n =$ size of $\mathcal{D}_n$).

Strategy 2) Draw without replacement $a_n \leq n$ observations into $\mathcal{D}_n$ (also called subsampling).

Exercise: for strategy 1, what is the probability that an observation $(\mathbf{x}_i, y_i)$ in $\mathcal{D}_n$ is present at least once in the drawn sample?

Reminder: Bagging = bootstrapping + aggregation.

Bagging algorithm with CART

Inputs:

- An original sample $\mathcal{D}_n$.
- A learning method/algorithm: here CART algorithm.
- Bagging parameters: B, a_n . - minnodes (size min to slice a node)

For $b = 1$ to B , repeat INDEPENDENTLY the two following steps:

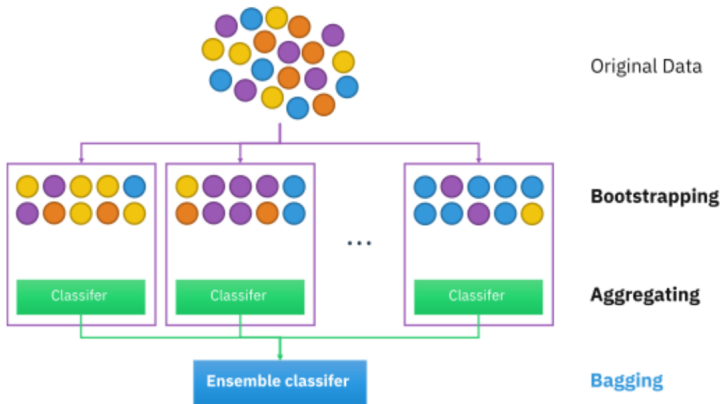
1. Draw a bootstrap sample $\mathcal{D}_n^{\theta_b}$ of size a_n from $\mathcal{D}_n$.
2. Build a large and unpruned CART tree using $\mathcal{D}_n^{\theta_b}$: get the predictor $f_{n,b}$.

end.

Output: The bagging prediction $\hat{f}_{n,B}(\mathbf{x})$ for a given observation $\mathbf{x}$

$$\hat{f}_{n,B}(\mathbf{x}) = \text{aggregation} \{f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})\}.$$

Bagging = bootstrapping + aggregation



- The prediction function $f_{n,b}$ built on the b -th bootstrap sample $\mathcal{D}_n^{\theta_b}$ depends on both $\mathcal{D}_n$ and θ_b , i.e.

$$f_{n,b} = f(., \mathcal{D}_n, \theta_b).$$

- The bagging prediction $\hat{f}_{n,B}(\mathbf{x})$ for the observation $\mathbf{x}$ is defined as the aggregation of the predictions $f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})$ over the B trees:

$$\hat{f}_{n,B}(\mathbf{x}) = \text{aggregation} \{f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})\}.$$

Bagging prediction in regression

Reminder: in regression, Y takes values in $\mathcal{Y} = \mathbb{R}$.

For an observation $\mathbf{x}$, the bagging prediction is the average of the predictions:

$$\hat{f}_{n,B}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_{n,b}(\mathbf{x}),$$

where $f_{n,b}(\mathbf{x})$ is the prediction of $\mathbf{x}$ for the b -th tree.

Bagging prediction in classification

Reminder: in classification, Y takes values in the finite and countable set $\mathcal{Y} = \{1, \dots, K\}$.

For an observation $\mathbf{x}$, the bagging prediction is the majority class over all the predictions, that means that $\hat{f}_{n,B}(\mathbf{x})$ is the label $\hat{k}$ that maximizes

$$\sum_{b=1}^B \mathbb{1}_{f_{n,b}(\mathbf{x})=k}, \quad \text{for all } k \in 1, \dots, K,$$

and where $f_{n,b}(\mathbf{x})$ is the prediction of $\mathbf{x}$ for the b -th tree.

Remark: $\sum_{b=1}^B \mathbb{1}_{f_{n,b}(\mathbf{x})=k}$ is the number of times the predicted label equal k over the B predictions.

Justification for bagging

Hypotheses:

- See $f_{n,1}, \dots, f_{n,B}$ as random variables ,
- that are identically distributed but not independent (two bootstrap samples have common observations (positive correlation)).

Bagging properties

Under the previous hypotheses, we can write

$$\mathbb{E}[\hat{f}_{n,B}(\mathbf{x})] = \mathbb{E}[f(\mathbf{x}, \theta, \mathcal{D}_n)] \quad (1)$$

Consequences: Bagging does not impact the bias.

→ Model selection: choose a learning procedure that construct a model with small bias $\Rightarrow$ large trees.

Hypotheses: random variables $f_{n,1}, \dots, f_{n,B}$ are identically distributed and positively correlated.

Bagging properties

Under the previous hypotheses, we can also write

$$\mathbb{V}[\hat{f}_{n,B}(\mathbf{x})] = \rho(\mathbf{x})\mathbb{V}[f(\mathbf{x}, \theta, \mathcal{D}_n)] + \frac{1 - \rho(\mathbf{x})}{B} \mathbb{V}[f(\mathbf{x}, \theta, \mathcal{D}_n)], \quad (2)$$

where $\rho(\mathbf{x}) = \text{corr}[f(\mathbf{x}, \theta_b, \mathcal{D}_n), f(\mathbf{x}, \theta_{b'}, \mathcal{D}_n)]$ with $b \neq b'$ and $\rho(\mathbf{x}) > 0$.

Consequences: Bagging reduces model variance and so improve model stability. Indeed, for large B ,

$$\mathbb{V}[\hat{f}_{n,B}(\mathbf{x})] \approx \rho(\mathbf{x})\mathbb{V}[f(\mathbf{x}, \theta, \mathcal{D}_n)].$$

→ So $\mathbb{V}[\hat{f}_{n,B}(\mathbf{x})] \leq \mathbb{V}[f(\mathbf{x}, \theta, \mathcal{D}_n)]$.

→ **Model selection:** variance reduction is more important for learning procedure sensitive to small change in the data: tree are good candidates.

Remark: conditionally to $\mathcal{D}_n$, random variables $f(\mathbf{x}, \theta_1, \mathcal{D}_n), \dots, f(\mathbf{x}, \theta_B, \mathcal{D}_n)$ are i.i.d.

Bagging properties

Under the previous hypotheses and **conditionally to $\mathcal{D}_n$** , we can also apply the law of large numbers and write

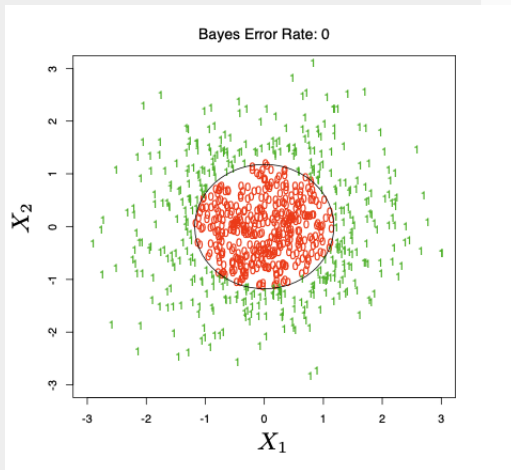
$$\begin{aligned}\lim_{B \rightarrow \infty} \hat{f}_{n,B}(\mathbf{x}) &= \lim_{B \rightarrow \infty} \frac{1}{B} \sum_{b=1}^B f(\mathbf{x}, \theta_b, \mathcal{D}_n) \\ &= \mathbb{E}_{\theta}[f(\mathbf{x}, \theta, \mathcal{D}_n)] \\ &= f_{\infty,n}(\mathbf{x}) \quad p.s. | \mathcal{D}_n,\end{aligned}$$

where $\mathbb{E}_{\theta}$ denotes the expectation computed according the distribution of θ and $f_{\infty,n}(\mathbf{x})$ is the bagging estimate (= limit of the bagging estimator).

→ **Interpretation:** As B increases, the bagging estimates stabilizes (predictions are stable).

→ **Choice of B :** choose B large to get stable estimates and to maximize the variance reduction.

- A simple deterministic problem.
- No noise: Bayes error=0.



→ We will compare performance of a single CART tree and bagging.

Example drawn from <https://web.stanford.edu/hastie/TALKS/boost.pdf>

Illustration with a toy example (2/5)

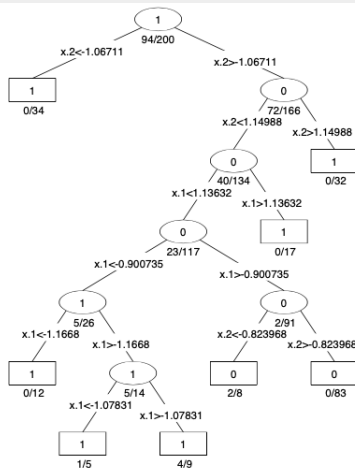


Figure 1: The single CART tree based on a train set of size 200.

Illustration: a toy example (3/5)

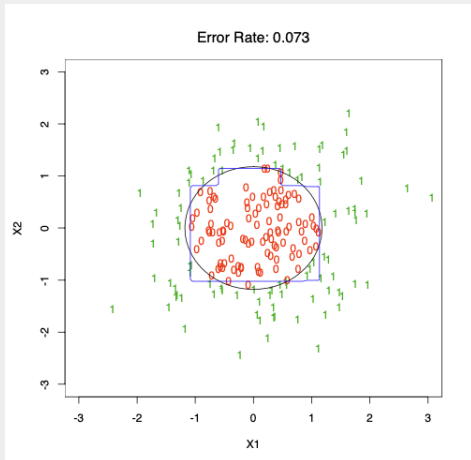


Figure 2: The partition associated with the previous single CART tree.

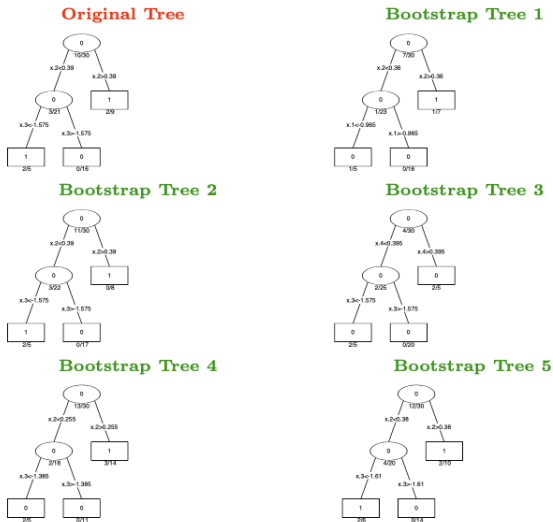


Figure 3: Single tree vs. Bagging.

Illustration: a toy example (5/5)

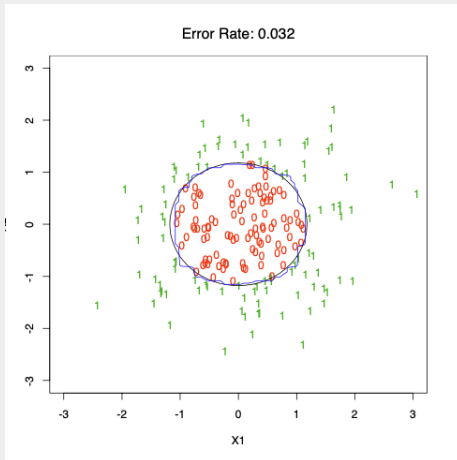


Figure 4: The partition obtained with bagging.

Illustration: a toy example (5/5)

- The decision boundaries are **smoother** (consequence of aggregation).
- **Bagging error is lower.**

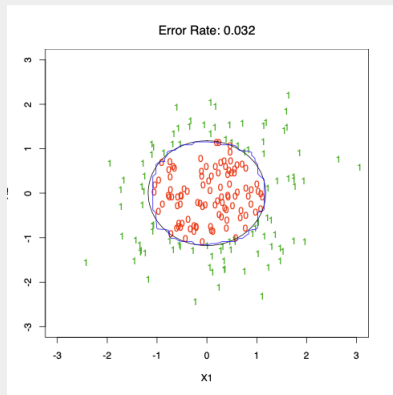


Figure 4: The partition obtained with bagging.

3.3 Random forests

3.3 Random forests



Author: Phill Cutler

Principle: Random forest = bagging + feature sampling.

Motivations:

- Same as bagging: performance improvement (by reducing variance) and no overfitting.
- Additional motivation: **greater variance reduction than bagging.**
→ **Strategy:** increase diversity among trees by using random feature sampling.

Justification for random forests

Bagging properties

- Bagging only affects the variance, it **reduces model variance and so improves model stability**.
- For large B , we have

$$\mathbb{V}[\hat{f}_{n,B}(\mathbf{x})] \approx \rho(\mathbf{x})\mathbb{V}[f(\mathbf{x}, \theta, \mathcal{D}_n)].$$

→ Random forests keeps the bagging properties.

→ **Random forests idea:** use feature sampling to reduce correlation $\rho(\mathbf{x})$ between trees.

→ **Conclusion:** Random forests achieves (generally) greater variance reduction than bagging by playing with $\rho(\mathbf{x})$.

Features sampling in random forests

- Modification of the CART algorithm.
- Features sampling performed during the tree building process and more precisely at each time we want to split a node:
 - (1) Random selection of $m < d$ features among the d features.
 - (2) Selection of the best split by using only this subset.

Remarks:

- let $\theta'_1, \dots, \theta'_b, \dots, \theta'_B$ be B independent random variables distributed as θ' and independent of $\mathcal{D}_n$.
 - θ'_b is used to draw features when the b -th tree is built (with $b = 1, \dots, B$).
- We can say that RF use rCART trees with a 'r' for random.

Reminder: Random forests = Bagging + Feature sampling.

RF algorithm

Inputs:

- An observation $\mathbf{x} \in \mathbb{R}^d$ that we want to predict.
- An original sample $\mathcal{D}_n$.
- RF parameters: $B, a_n, m, \text{nodesize}$.

For $b = 1$ to B , repeat INDEPENDENTLY the two following steps:

1. Draw a bootstrap sample $\mathcal{D}_n^{\theta_b}$ of size a_n from $\mathcal{D}_n$.
2. Build a large and unpruned rCART tree on $\mathcal{D}_n^{\theta_b}$ by repeating the following steps on all nodes until all nodes has `nodesize` or less observations.
To split a node,
 - a) Draw randomly m features.
 - b) Split the node according to the best split based only on the m selected features.

end.

Output: The RF prediction $\hat{f}_n^{\text{RF}}(\mathbf{x})$ for the observation $\mathbf{x}$

$$\hat{f}_{n,B}(\mathbf{x}) = \text{aggregation} \{f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})\}.$$

Random forests = Bagging + Features sampling

Bootstrap sampling

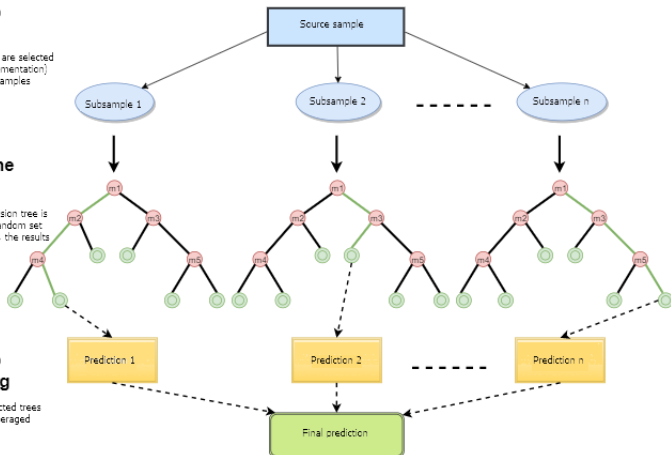
r (percentage) examples are selected
(0.63 in classical implementation)
in n random subsamples

Building the models

for each subsample, a decision tree is
constructed based on a random set
of m features (covariates); the results
fall into leaves

Bootstrap aggregating

results from all constructed trees
are gathered and averaged



- The prediction function $f_{n,b}$ of the b -th tree depends on $\mathcal{D}_n$, θ_b and θ'_b , i.e.

$$f_{n,b} = f(., \mathcal{D}_n, \theta_b, \theta'_b).$$

- The bagging prediction $\hat{f}_n^{\text{Bag}}(\mathbf{x})$ is defined as the aggregation of the predictions $f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})$ over the B trees

$$\hat{f}_{n,B}(\mathbf{x}) = \text{aggregation} \{f_{n,1}(\mathbf{x}), \dots, f_{n,B}(\mathbf{x})\}.$$

RF prediction in regression

Reminder: in regression, Y takes values in $\mathbb{R}$.

For an observation $\mathbf{x}$, the RF prediction is the average of the predictions:

$$\hat{f}_{n,B}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_{n,b}(\mathbf{x}),$$

where $f_{n,b}(\mathbf{x})$ is the prediction of $\mathbf{x}$ for the b -th tree.

RF prediction in classification

Reminder: in classification, Y takes values in the set of labels $1, \dots, K$.

For an observation $\mathbf{x}$, the RF prediction is the majority class over all the predictions, that means that $\hat{f}_{n,B}(\mathbf{x})$ is the label $\hat{k}$ that maximizes

$$\sum_{b=1}^B \mathbb{1}_{f_{n,b}(\mathbf{x})=k}, \quad \text{for all } k \in 1, \dots, K,$$

and where $f_{n,b}(\mathbf{x})$ is the prediction of $\mathbf{x}$ for the b -th tree.

Remark: $\sum_{b=1}^B \mathbb{1}_{f_{n,b}(\mathbf{x})=k}$ is the number of times the predicted label equal k over the B predictions.

Parameters in RF:

- B = the number of trees in the forest.
→ Take B large (default value: $B = 500$) to obtain stable/robust estimate.
- $a_n \leq n$ = the size of each bootstrap sample and the bootstrap strategie.
→ Two usual bagging strategies:
 - (1) Choose $a_n = n$ and draw with replacement,
 - (2) Choose $a_n \leq n$ and draw without replacement.
- `nodesize` = the minimum number of observations required to split a node.
→ Take `nodesize` small to obtain trees with **small bias** and **large variance**.
- `m` = the number of features randomly selected when splitting a node.
→ **Almost the most important parameter to be tuned.**

What value for `max_features` ?

Parameter `m`: *"slightly controls"* the bias-variance tradeoff for the forest.

Explanation:

- **Smaller `max_features`:** more different and less correlated trees (smaller $|\rho|$) but tree that do not fit well the data (larger bias).
- **Larger `max_features`:** trees that fit correctly the data (= smaller bias) but tree more similar and so more correlated (larger $|\rho|$).

Default values: $m = d/3$ in regression and $m = \sqrt{d}$ in classification.

→ Often reported to be good choices.

→ Yet, optimal value for `m` depends mainly on the data: use of the out-of-bag samples to tune this parameter.

3.4 Out-Of-Bag sample

3.4 Out-Of-Bag sample

As for other machine learning algorithms, we need a **criterion to measure performances of a random forest**.

Most common criteria (reminder):

- The quadratic error for regression: $\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{f}_n^{\text{RF}}(\mathbf{x}_i))^2$.
- The misclassification error for classification: $\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{Y_i \neq \hat{f}_n^{\text{RF}}(\mathbf{x}_i)}$.

Reminder: to be a good proxy of the *true* error, these criteria must be **evaluated on a test set or by using cross validation, leave-one-out, etc.**

Bootstrapping-step asset: performance criteria can be evaluated directly on the original dataset $\mathcal{D}_n$ by using the Out of Bag samples (OOB).

→ With RF (or bagging): no need for a test set or to use strategies such as cross validation or Monte Carlo approaches)

3.4 Out-Of-Bag sample

As for other machine learning algorithms, we need a criterion to measure performances of a random forest.

Most common criteria (reminder):

- The quadratic error for regression: $\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{f}_n^{\text{RF}}(\mathbf{x}_i))^2$.
- The misclassification error for classification: $\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{Y_i \neq \hat{f}_n^{\text{RF}}(\mathbf{x}_i)}$.

Reminder: to be a good proxy of the *true* error, these criteria must be evaluated on a test set or by using cross validation, leave-one-out, etc.

Bootstrapping-step asset: performance criteria can be evaluated directly on the original dataset $\mathcal{D}_n$ by using the Out of Bag samples (OOB).

→ With RF (or bagging): no need for a test set or to use strategies such as cross validation or Monte Carlo approaches)

Bootstrapping step (reminder)

- **Input:** a data set $\mathcal{D}_n$ with 8 observations

$\mathcal{D}_n$	obs 1	obs 2	obs 3	obs 4	obs 5	obs 6	obs 7	obs 8
-----------------	-------	-------	-------	-------	-------	-------	-------	-------

- **Do bootstrapping:** Draw observations randomly and with replacement into $\mathcal{D}_n$. Repeat independently this procedure B times.

$\mathcal{D}_n^1$	obs 2	obs 6	obs 5	obs 5	obs 2	obs 4	obs 8	obs 7
$\mathcal{D}_n^2$	obs 6	obs 8	obs 2	obs 4	obs 2	obs 7	obs 8	obs 1
$\vdots$	$\vdots$							
$\mathcal{D}_n^B$	obs 7	obs 8	obs 3	obs 7	obs 2	obs 3	obs 3	obs 1

→ **Output:** B bootstrap-resampled versions of the original $\mathcal{D}_n \Leftrightarrow$ B "new" data sets made up of observations of $\mathcal{D}_n$

$$\mathcal{D}_n^1, \dots, \mathcal{D}_n^B.$$

Bootstrapping step

- **Input:** a data set $\mathcal{D}_n$ with 8 observations

$\mathcal{D}_n$	obs 1	obs 2	obs 3	obs 4	obs 5	obs 6	obs 7	obs 8
-----------------	-------	-------	-------	-------	-------	-------	-------	-------

- **Do bootstrapping:** obtain B bootstrap samples

$\mathcal{D}_n^1$	obs 2	obs 6	obs 5	obs 5	obs 2	obs 4	obs 8	obs 7
$\mathcal{D}_n^2$	obs 6	obs 8	obs 2	obs 4	obs 2	obs 7	obs 8	obs 1
$\vdots$	$\vdots$							
$\mathcal{D}_n^B$	obs 7	obs 8	obs 3	obs 7	obs 2	obs 3	obs 3	obs 1

- **B OOB samples:** observations in $\mathcal{D}_n$ that are not in the B bootstrap samples.

$OOB^1 :$	obs 1	obs 3	obs 6	obs 6	obs 6	obs 6	obs 6	ind 6
$OOB^2 :$	obs 3	obs 5						
$\vdots$	$\vdots$							
$OOB^B :$	obs 4	obs 5	obs 6					

→ Use the OOB samples to estimate the error of a random forest.

Notations:

- f_n^b : the prediction function associated to the b -th tree.
- OOB^b : the b -th out-of-bag sample, formally $OOB^b = \mathcal{D}_n \setminus \mathcal{D}_n^b$.
- n_{OOB^b} : size of OOB^b .

OOB error of the b -th tree

The **OOB error of the b -th tree** is the error of the b -th tree estimated on the b -th out-of-bag sample OOB^b ,

- **In regression:** $E_{OOB^b} = \frac{1}{n_{OOB^b}} \sum_{i \in OOB^b} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
- **In classification:** $E_{OOB^b} = \frac{1}{n_{OOB^b}} \sum_{i \in OOB^b} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{x}_i)}$.

OOB error of a random forest

The OOB error of a random forest is the average of the OOB error over all the trees of the forest,

$$err_{OOB} = \frac{1}{B} \sum_{b=1}^B E_{OOB^b}.$$

Notations:

- f_n^b : the prediction function associated to the b -th tree.
- OOB^b : the b -th out-of-bag sample, formally $OOB^b = \mathcal{D}_n \setminus \mathcal{D}_n^b$.
- n_{OOB^b} : size of OOB^b .

OOB error of the b -th tree

The **OOB error of the b -th tree** is the error of the b -th tree estimated on the b -th out-of-bag sample OOB^b ,

- **In regression:** $E_{OOB^b} = \frac{1}{n_{OOB^b}} \sum_{i \in OOB^b} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
- **In classification:** $E_{OOB^b} = \frac{1}{n_{OOB^b}} \sum_{i \in OOB^b} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{x}_i)}$.

OOB error of a random forest

The OOB error of a random forest is the average of the OOB error over all the trees of the forest,

$$err_{OOB} = \frac{1}{B} \sum_{b=1}^B E_{OOB^b}.$$

3.5 Variable importance

3.5 Variable importance

One major asset of the single trees: models highly interpretable (when trees are not too complex).

A random forest = aggregation of several single trees.

→ Not directly interpretable.

Alternative with RF: introduction of an importance score that measures the importance of each feature in the algorithm for predicting the target.

→ Importance score can be used to rank features by decreasing order of their importance in the algorithm prediction process. → And so to perform features selection.

→ We will focus here on the Mean Decrease in Accuracy (MDA).

Value of the MDA for the j -th feature, $MDA(\mathbf{X}^j)$

Calculation of $MDA(\mathbf{X}^j)$

Input: a forest, $\mathcal{D}_n$ and the j -th feature $\mathbf{X}^j$.

For $b = 1$ to B do:

- **Step 1:** Get the b -th OOB sample, OOB^{θ_b} and compute the OOB error of the b -th tree:
 - **In regression:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.
- **Step 2:** Permute randomly values of $\mathbf{X}^j$ in OOB^{θ_b} : get the permuted OOb sample denoted by $\widetilde{OOB}_j^{\theta_b}$.
- **Step 3:** compute the OOB error of the b -th tree on $\widetilde{OOB}_j^{\theta_b}$:
 - **In regression:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.

End for.

Output: $MDA(\mathbf{X}^j) = \frac{1}{B} \sum_{b=1}^B \left[E_{\widetilde{OOB}_j^{\theta_b}} - E_{OOB^{\theta_b}} \right]$.

Value of the MDA for the j -th feature, $MDA(\mathbf{X}^j)$

Calculation of $MDA(\mathbf{X}^j)$

Input: a forest, $\mathcal{D}_n$ and the j -th feature $\mathbf{X}^j$.

For $b = 1$ **to** B **do:**

- **Step 1:** Get the b -th OOB sample, OOB^{θ_b} and compute the OOB error of the b -th tree:
 - **In regression:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.
- **Step 2:** Permute **randomly** values of $\mathbf{X}^j$ in OOB^{θ_b} : get the permuted OOb sample denoted by $\widetilde{OOB}_j^{\theta_b}$.
- **Step 3:** compute the OOB error of the b -th tree on $\widetilde{OOB}_j^{\theta_b}$:
 - **In regression:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.

End for.

Output: $MDA(\mathbf{X}^j) = \frac{1}{B} \sum_{b=1}^B \left[E_{\widetilde{OOB}_j^{\theta_b}} - E_{OOB^{\theta_b}} \right]$.

Value of the MDA for the j -th feature, $MDA(\mathbf{X}^j)$

Calculation of $MDA(\mathbf{X}^j)$

Input: a forest, $\mathcal{D}_n$ and the j -th feature $\mathbf{X}^j$.

For $b = 1$ to B do:

- **Step 1:** Get the b -th OOB sample, OOB^{θ_b} and compute the OOB error of the b -th tree:
 - **In regression:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{OOB^{\theta_b}} = \frac{1}{|OOB^{\theta_b}|} \sum_{i \in OOB^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.
- **Step 2:** Permute **randomly** values of $\mathbf{X}^j$ in OOB^{θ_b} : get the permuted OOb sample denoted by $\widetilde{OOB}_j^{\theta_b}$.
- **Step 3:** compute the OOB error of the b -th tree on $\widetilde{OOB}_j^{\theta_b}$:
 - **In regression:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} (Y_i - f_{n,b}(\mathbf{X}_i))^2$.
 - **In classification:** $E_{\widetilde{OOB}_j^{\theta_b}} = \frac{1}{|\widetilde{OOB}_j^{\theta_b}|} \sum_{i \in \widetilde{OOB}_j^{\theta_b}} \mathbb{1}_{Y_i \neq f_{n,b}(\mathbf{X}_i)}$.

End for.

Output: $MDA(\mathbf{X}^j) = \frac{1}{B} \sum_{b=1}^B \left[E_{\widetilde{OOB}_j^{\theta_b}} - E_{OOB^{\theta_b}} \right]$.

The importance of the j -th feature is:

$$MDA(\mathbf{X}^j) = \frac{1}{B} \sum_{b=1}^B \left[\widetilde{E_{OOB_j^{\theta_b}}} - E_{OOB^{\theta_b}} \right].$$

- **Meaning:** the importance score of the j -th feature is defined as the difference between the average error of the trees on the OOB samples and the average error of the trees on the permuted OOB samples.
- **Explanation:** Permutations are used to mimics independence (no link) between $\mathbf{X}^j$ and Y .
→ The feature $\mathbf{X}^j$ is considered as important in the algorithm prediction process if by breaking the link between $\mathbf{X}^j$ and Y , the error of the forest increases.
- **Interpretation:** large difference between the two OOB errors $\Rightarrow$ large $MDA(\mathbf{X}^j) \Rightarrow \mathbf{X}^j$ is an important feature in the forest to predict Y .
- ⚠ Several definitions of MDA : depends on the software implementation (library Scikit-Learn $\neq$ libraries ranger and randomForest in R).
→ MDA computed on an independant set in Scikit-Learn.

The importance of the j -th feature is:

$$MDA(\mathbf{X}^j) = \frac{1}{B} \sum_{b=1}^B \left[\widetilde{E_{OOB_j^{\theta_b}}} - E_{OOB^{\theta_b}} \right].$$

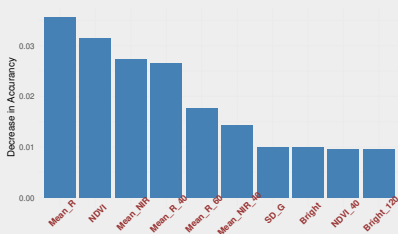
- **Meaning:** the importance score of the j -th feature is defined as the difference between the average error of the trees on the OOB samples and the average error of the trees on the permuted OOB samples.
- **Explanation:** Permutations are used to mimics independence (no link) between $\mathbf{X}^j$ and Y .
→ The feature $\mathbf{X}^j$ is considered as important in the algorithm prediction process if by breaking the link between $\mathbf{X}^j$ and Y , the error of the forest increases.
- **Interpretation:** large difference between the two OOB errors $\Rightarrow$ large $MDA(\mathbf{X}^j) \Rightarrow \mathbf{X}^j$ is an important feature in the forest to predict Y .
- **⚠ Several definitions of MDA : depends on the software implementation** (library Scikit-Learn $\neq$ libraries ranger and randomForest in R).
→ MDA computed on an independant set in Scikit-Learn.

MDA: example on the land cover data set

We build a frandom forest and we want to know what are the 10 most importance features in the built random forest.

→ We decide to compute MDA for each feature. Formally, we compute independently MDA several times on the test set and then we estimate the average.

→ The 10 variables with the highest MDA average are plotted below.



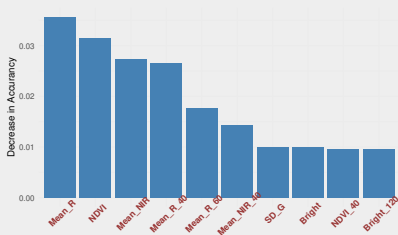
According to the MDA criterion, the most importance feature to explain the land cover type in the forest is the feature Mean_R following by the variable NDVI.

MDA: example on the land cover data set

We build a frandom forest and we want to know what are the 10 most importance features in the built random forest.

→ We decide to compute MDA for each feature. Formally, we compute independently MDA several times on the test set and then we estimate the average.

→ The 10 variables with the highest MDA average are plotted below.



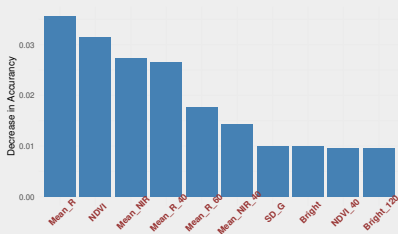
According to the MDA criterion, the most importance feature to explain the land cover type in the forest is the feature Mean_R following by the variable NDVI.

MDA: example on the land cover data set

We build a frandom forest and we want to know what are the 10 most importance features in the built random forest.

→ We decide to compute MDA for each feature. Formally, we compute inde-
pendently MDA several times on the test set and then we estimate the average.

→ The 10 variables with the highest MDA average are plotted below.



According to the MDA criterion, the most importance feature to explain the land cover type in the forest is the feature Mean_R following by the variable NDVI.

Other importance variable indexes that can be used with random forests:

- ▷ **Mean decrease in impurity (MDI)**: consists for each tree in summing up the impurity gain associated to all splits of the tree performed along a given variable and then average it over all trees of the forests.
- ▷ **SHAP**: to compute the Shapley values from the Random Forest forests. The Shapley value comes from cooperative game theory and measures the marginal contribution of each feature when combined with other features.
- ▷ **SobolMDA**: an importance measure based on the sobol index and adapted to random forests [Bénard et al., 2022b].
- ▷ **SHAFF**: a fast Shapley effect estimate for random forests [Bénard et al., 2022a].

Sometimes, we don't want to use all the feature and so we want to select a subset of relevant variables to be used to built our model. **Why ?**

- ▷ **Overfitting**: for some models, if there are more features than the number of samples, The model cannot be correctly estimated (example: in linear model).
- ▷ **Interpretation**: we want our models to be simple and explainable. We lose explainability when we have a lot of features.
- ▷ **Improve performance**: most of the times, there many non-informative features and for some models non-informative features or redundant information can affect model performance.
- ▷ **Other reasons**: fitting a model with too much features can be time-consuming, harder to implement in production.

Sometimes, we don't want to use all the feature and so we want to select a subset of relevant variables to be used to built our model. **Why ?**

- ▷ **Overfitting**: for some models, if there are more features than the number of samples, The model cannot be correctly estimated (example: in linear model).
- ▷ **Interpretation**: we want our models to be simple and explainable. We lose explainability when we have a lot of features.
- ▷ **Improve performance**: most of the times, there many non-informative features and for some models non-informative features or redundant information can affect model performance.
- ▷ **Other reasons**: fitting a model with too much features can be time-consuming, harder to implement in production.

There exist several algorithms that automates the variable selection, some examples available in R (also available in Python):

- ▷ Boruta
(<https://cran.r-project.org/web/packages/Boruta/Boruta.pdf>)
- ▷ Recursive feature elimination (REF) (an RFE algorithm with random forests:
<https://cran.r-project.org/web/packages/VSURF/index.html>)
- ▷ Recursive feature elimination with crossvalidation (REFCV)
(https://search.r-project.org/CRAN/refmans/mlr3fselect/html/mlr_fselectors_rfecv.html)

You can also implement our own variable selection algorithm. To do so you need to :

1. Choose a variable importance measure
2. Define the algorithm principle, for instance:
 - Select the P most important features based on the chosen importance criterion,
 - Implement a RFE method.

- Random Forest = an improvement over bagged CART trees.
- Algorithm with few parameters to tune.
- No overfitting problem compared to a single tree.
- Higher performances than a single CART tree or bagged CART trees.
- No need of test set: out-of-bag samples.
- Interpretation tool: variable importance score.
- Large applicability.



Bénard, C., Biau, G., Da Veiga, S., and Scornet, E. (2022a).

Shaff: Fast and consistent shapley effect estimates via random forests.

In *International Conference on Artificial Intelligence and Statistics*, pages 5563–5582. PMLR.



Bénard, C., Da Veiga, S., and Scornet, E. (2022b).

Mean decrease accuracy for random forests: inconsistency, and a practical solution via the sobol-mda.

Biometrika, 109(4):881–900.



Breiman, L. (1996).

Bagging predictors.

Machine learning, 24(2):123–140.



Breiman, L. (2001).

Random forests.

Machine learning, 45(1):5–32.



Freund, Y., Schapire, R. E., et al. (1996).

Experiments with a new boosting algorithm.

In *icml*, volume 96, pages 148–156. Citeseer.